Snip

Short links with live analytics

MERN STACK EDITION | PROJECT 1 FROM THE FRESHER PROJECTS GUIDE

Step-by-step: install, run, test, demo, deploy,
and turn it into strong resume bullets.

React | Express | MongoDB | Redis | BullMQ | Socket.IO

1. What you are building

Snip is a link shortener. You paste a long link, get a short one like `localhost:5000/abc1234`, and every click appears on a dashboard **live**: where the person is from, which device, which browser, and which app they came from (Instagram, WhatsApp...).

This is Project 1 of the guide, rebuilt to fit **your MERN stack**. The guide used Next.js, Fastify and PostgreSQL. Everything is swapped to what you already know, and nothing outside MERN is required except Redis (needed for the cache and queue, and it is optional to start).

Guide used	You will use	Why this is fine
Next.js + Tailwind + Recharts	React (Vite) + Tailwind + Recharts	Same UI result, plain React
Node.js + Fastify	Node.js + Express	Your main backend
PostgreSQL	MongoDB + Mongoose	Your main database. Click analytics use aggregation pipelines (you already know these)
Redis cache	Redis cache (ioredis)	Same. Falls back to memory if you skip Redis
BullMQ queue + worker	BullMQ queue + worker	Same
Socket.IO	Socket.IO	Same
MaxMind GeoLite2	geoip-lite (offline IP database)	No account or license key needed
k6 + Docker Compose	k6 + Docker Compose	Same

Why this is NOT just another URL shortener

- A basic shortener only stores *slug -> URL*. Snip adds the hard parts that interviewers ask about: **caching**, an **async queue**, **real-time** updates, and **load testing**.
- Redirects never wait for the database. The click is queued and processed in the background.
- Extra touches that make it yours: password-protected links, expiring links, QR codes, bot filtering, hashed IPs, rate limiting and a live click feed.

> **Be ready to explain the difference** from a simple Express + MongoDB shortener. Your answer: "mine serves redirects from a Redis cache, processes clicks through a queue, and streams analytics live."

What each feature maps to in the code

Feature	File
Fast redirect + cache + queue push	server/src/routes/redirect.js, services/cache.js
Queue and worker	server/src/services/queue.js, services/clickProcessor.js
Country / device / browser / bot filtering / IP hash	server/src/services/clickProcessor.js
Analytics (aggregation pipeline)	server/src/routes/links.js (GET /:id/stats)
Live updates	server/src/socket.js + client/src/hooks/useLiveClicks.js
Dashboard UI	client/src/pages/LinkDetail.jsx
Load test	loadtest/redirect.js

2. What to install first

Tool	Needed?	How to check
Node.js 18 or newer (20 LTS best)	Yes	node -v
VS Code	Yes	-
MongoDB: Docker Desktop **or** MongoDB Atlas (free) **or** local MongoDB	Yes (one of them)	see Step 2
Redis: Docker (same compose file)	Recommended	optional to start
Git	For GitHub	git --version
k6 (load testing)	Later	k6 version

Windows tip: Redis does not run natively on Windows. Easiest fix: Docker Desktop (Step 2, Option A). If you do not want Docker, use MongoDB Atlas or your local MongoDB and leave `REDIS_URL` empty. The project still works with an in-memory cache and queue.

Project folder

```
shortlinks/
  docker-compose.yml        <- starts MongoDB + Redis
  README.md
  loadtest/redirect.js      <- k6 load test
  server/                   <- Express API + worker + sockets
    .env.example
    package.json
    scripts/seed.js         <- demo data
    src/
      index.js              <- starts everything
      config.js
      socket.js
      models/    User.js  Link.js  Click.js
      middleware/ auth.js  rateLimit.js
      routes/    auth.js  links.js  redirect.js
      services/  cache.js  queue.js  redis.js  clickProcessor.js
      utils/pages.js        <- 404 / expired / password pages
  client/                   <- React app (Vite + Tailwind)
    src/
      App.jsx  main.jsx  api.js  utils.js  index.css
      context/AuthContext.jsx
      hooks/useLiveClicks.js
      pages/     AuthPage.jsx  Links.jsx  LinkDetail.jsx
      components/ Layout.jsx  CreateLinkForm.jsx  LiveFeed.jsx  QRBox.jsx
```

3. Run it, step by step

STEP 1 Open the project in VS Code

- Unzip `shortlinks.zip`.
- VS Code -> **File -> Open Folder** -> choose the `shortlinks` folder.
- Open the terminal: **Terminal -> New Terminal** (shortcut Ctrl + `). You will use **three terminals**: one for Docker, one for server, one for client. Use the **+** button for more.

STEP 2 Start MongoDB and Redis (pick ONE option)

Option A: Docker Desktop (recommended)

```
# in the shortlinks folder
docker compose up -d
docker ps            # you should see snip-mongo and snip-redis
```

MongoDB is now at `mongodb://127.0.0.1:27017` and Redis at `redis://127.0.0.1:6379`. These are already the defaults in `.env.example`.

Option B: MongoDB Atlas, no Redis

- Create a free cluster at mongodb.com/atlas, create a database user, and under **Network Access** allow your IP.
- Click **Connect -> Drivers** and copy the connection string. Put it in `MONGO_URI` in Step 3 (add `/shortlinks` before the ?).
- Set `REDIS_URL=` (empty). The app runs with an in-memory cache and queue. Good enough to build and test; use Redis for your final demo and load test.

Option C: MongoDB already installed on your PC: start it and keep the default `MONGO_URI`. Leave `REDIS_URL` empty.

STEP 3 Set up and start the backend

```
cd server
copy .env.example .env        # Mac/Linux: cp .env.example .env
npm install
npm run dev
```

Open `server/.env` and check it:

Variable	What to put
MONGO_URI	Default works for Docker/local. Atlas: paste your string
REDIS_URL	`redis://127.0.0.1:6379` with Docker, or empty to run without Redis
JWT_SECRET, IP_SALT	Any long random text. Change them before deploying
DEMO_GEO	true for local testing (see note below)

You should see:

```
[mongo] connected
[http] listening on http://localhost:5000  (cache: redis, queue: bullmq)
```

Check `http://localhost:5000/api/health` in the browser. It should show `{"ok":true,"cache":"redis","queue":"bullmq","mongo":true}`.

> **About DEMO_GEO:** when you click from your own PC, your IP is local (127.0.0.1), which has no country. With `DEMO_GEO=true` local clicks get a random country so the dashboard looks alive. **Set it to false in production** and never present those countries as real data. Real visitors are located by their real IP.

STEP 4 Add demo data (optional but great for screenshots)

```
# in the server folder, backend can keep running in another terminal
npm run seed
```

This creates the account **demo@example.com / demo1234** with three links and about 1,500 fake clicks over 14 days.

STEP 5 Start the frontend

Open a **new terminal** (the server must keep running):

```
cd client
npm install
npm run dev
```

Open `http://localhost:5173`. In development the React app talks to the API through a proxy (see `vite.config.js`), so you do not need any CORS setup.

STEP 6 Use the app and see the live dashboard

- Log in with the demo account, or sign up with your own.
- Paste a long URL (for example a YouTube video), click **Shorten**. Try **More options** for expiry and password.
- Click the short link (`localhost:5000/xxxxxxx`). You land on the real site.
- Open **Analytics** for that link. Open the short link in another tab again: the **live feed** and counters update without refreshing.
- Scan the QR code with your phone to test it (the phone must be on the same Wi-Fi, see Step 7).

STEP 7 Test from your phone (same Wi-Fi)

- Find your PC's local IP: run `ipconfig` (Windows) and read **IPv4 Address**, for example 192.168.1.8.
- In `server/.env` set `BASE_URL=http://192.168.1.8:5000`, restart the server, create a new link.
- Open that short link on your phone. Allow port 5000 through the Windows firewall if asked.
- Your phone is a private IP too, so with DEMO_GEO the country is still random. Real country detection needs a deployed app (Section 6).

STEP 8 Load test with k6 (this gives your real resume number)

- Install k6 from k6.io (Windows: `winget install k6`).
- Create a link and note its slug (the part after the slash).

```
k6 run -e SLUG=menu-demo loadtest/redirect.js
```

- Read **http_reqs** (requests/second) and **http_req_duration p(95)**. Write them in the README table.
- Run it against the **deployed** app too, and note both results. Never claim a number you did not measure.
- The goal in the original guide is 10,000 clicks per second. That is a stretch goal: it needs good hardware and tuning. Report what you actually get; an honest 800 req/s that you can explain beats a made-up 10,000.

> **Tip:** after the load test, look at `/api/health` and the dashboard. If the queue is behind, clicks appear a few seconds late. That is normal and is exactly what the queue is for. It is also a great interview story.

4. How it works (know this for interviews)

The life of one click

```
1. Visitor opens  /abc1234
2. Redirect route looks in Redis:   link:abc1234  ->  { url, id, expiry, hasPassword }
     miss?  read MongoDB once, store in Redis for 1 hour
3. Expired?  -> 410 page.   Password?  -> password form.   Otherwise:
4. Push { linkId, ip, userAgent, referrer, time } to the BullMQ queue   (NOT awaited)
5. Answer HTTP 302 redirect immediately
------------------------------------------------------------------------
6. Worker takes the job:  skip bots -> GeoIP lookup -> parse user agent
7. Save Click in MongoDB (IP stored only as salted hash), +1 on the link counter
8. Emit 'click' over Socket.IO to room user:<ownerId>
9. Dashboard receives it: feed updates instantly, charts refresh (debounced)
```

Important decisions and how to defend them

Decision	Why
302 redirect, not 301	Browsers cache 301 forever, so repeat visits would skip the server and never be counted
Queue instead of saving in the redirect	Redirect speed stays constant even when MongoDB is slow or busy
Redis cache in front of MongoDB	A redirect is a read-heavy key lookup: perfect cache use case
Cache errors fall back to MongoDB	Cache is an optimization, never a single point of failure
Delete cache key when a link is deleted	Otherwise a deleted link keeps working until the cache expires
Hashed IP (salted SHA-256)	Privacy: you can count unique visitors without storing personal data
Unique index on slug; index on (linkId, ts)	Instant slug lookup; fast time-range analytics
Private Socket.IO room per user	Users only receive their own clicks
Rate limit login, signup, link creation, password tries	Stops abuse and password guessing
Bot filtering (isbot)	WhatsApp/Google link-preview bots would otherwise inflate click counts

5. Test checklist

#	Test	Expected
1	Sign up with a new email, then log out and log in	Works; wrong password shows an error
2	Create a link with no slug	Random 7-character slug
3	Create a link with a custom slug twice	Second time: 'That slug is already taken'
4	Create a link with url ftp://x.com	Rejected (only http/https)
5	Open the short link	Redirects to the long URL
6	Open Analytics, click the short link again	Live feed row + counter update without refresh
7	Create a link with a password, open it	Password page; wrong password rejected, right one redirects
8	Create a link that expires in 2 minutes, wait	Shows 'Link expired' (HTTP 410)
9	Delete a link, open its short URL	404 'Link not found' (cache cleared)

#	Test	Expected
10	Stop Redis (docker stop snip-redis), open a link	Redirect still works (falls back to MongoDB). Good demo of resilience
11	Log in as another user	Cannot see the first user's links or live clicks

6. Make it live (this is what makes it a resume project)

> The guide's rule: **a live URL, a clear README and a real demo beat five half-finished repos.** Deploy it, put the link at the top of your resume.

Free deployment plan

Part	Where	Notes
Database	MongoDB Atlas (free M0)	Allow access from anywhere (0.0.0.0/0) for Render, use a strong password
Redis	Redis Cloud free tier or Render Key Value	BullMQ polls often. Upstash's pay-per-command free tier can run out quickly, so prefer a fixed-size free Redis
Backend	Render (Web Service) or Railway	Root directory: server. Build: npm install. Start: npm start. Supports WebSockets
Frontend	Vercel or Netlify	Root directory: client. Build: npm run build. Output: dist. `vercel.json` is already included for routing

Environment variables in production

Where	Variable	Value
Render (server)	MONGO_URI, REDIS_URL	Your Atlas string and Redis URL
Render (server)	JWT_SECRET, IP_SALT	Long random strings
Render (server)	BASE_URL	Your backend URL, e.g. https://snip-api.onrender.com (short links will look like this)
Render (server)	CLIENT_URL	Your frontend URL (needed for CORS and sockets)
Render (server)	TRUST_PROXY	1 (so the real visitor IP is read behind Render's proxy)
Render (server)	DEMO_GEO	false
Vercel (client)	VITE_API_URL	Your backend URL

> **Free-tier warning:** Render's free web service sleeps when idle, so the first click after a pause can take about 30 seconds. Open the app a minute before showing it to anyone. A custom domain (even a cheap one) makes short links look real: set BASE_URL to it.

Show it in your 60-second demo

- Create a link, open it on your phone (scan the QR code).
- Show the dashboard updating live on your laptop.
- Show the k6 result and say what the number means.
- Show the architecture diagram from the README.

Resume bullets (replace [X] with numbers you measured)

- Built a full-stack link-shortening platform (React, Node/Express, MongoDB, Redis) with Redis-cached redirects at **[X] ms** p95 latency and a live click dashboard over WebSockets.
- Designed an async click pipeline (BullMQ queue + worker) that handled **[X] redirects/second** in k6 load tests while enriching each click with country, device and referrer.
- Implemented password-protected and expiring links, QR codes, bot filtering, IP hashing and rate limiting; deployed on [Render + Atlas + Vercel] with a live demo.

Resume project title: Snip - Short Links with Live Analytics | *React, Node.js, Express, MongoDB, Redis, BullMQ, Socket.IO, Docker, k6*

Keep your older "URL Shortener" project off the resume or merge it into this one. Two shorteners look repetitive; one strong version looks focused.

7. Troubleshooting

Problem	Fix
`Failed to start: connect ECONNREFUSED 127.0.0.1:27017`	MongoDB is not running. Run `docker compose up -d` or fix MONGO_URI
Terminal spams Redis connection errors	Redis is not running. Start Docker, or set `REDIS_URL=` (empty) in server/.env and restart
Atlas: `MongooseServerSelectionError`	Add your IP in Network Access; check username/password (special characters in the password must be URL-encoded)
Frontend shows 'Failed to fetch' / blank	Backend is not running on port 5000. Check the server terminal
Port 5000 already in use (Mac: AirPlay)	Set PORT=5001 in server/.env, update BASE_URL, and the proxy target in client/vite.config.js
LIVE badge stays on 'connecting...'	Log out and in again (token), confirm the server is up, and in production check CLIENT_URL matches your frontend exactly (no trailing slash)
Clicks not appearing	Bots are skipped: curl and link-preview user agents may count as bots. Test in a normal browser tab. Check the server terminal for worker errors
Every country is random	Expected locally with DEMO_GEO=true. Deploy with DEMO_GEO=false for real countries
Deleted or edited link still redirects	Cache holds it up to 1 hour. Deleting clears the key; if you edit a document manually in Mongo, flush Redis: `docker exec snip-redis redis-cli flushall`
`npm install` fails on geoip-lite	Use Node 18/20, delete node_modules and package-lock.json, run npm install again
PowerShell blocks npm scripts	Run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` or use Command Prompt

8. Interview questions you should be able to answer

Why a queue for clicks?

So the redirect never waits for a database write. The user gets the redirect in a few ms and analytics are processed in the background. If MongoDB is slow, redirects are unaffected, and BullMQ retries failed jobs.

What happens if Redis goes down?

Cache reads fail fast and fall back to MongoDB, so redirects keep working, just slower. The click is added to the queue without awaiting it, so the redirect is not blocked, but those click events are delayed or lost until Redis is back. A next step is a fallback that buffers them.

How do you generate slugs and avoid collisions?

Random 7 characters from a 56-letter alphabet (about 1.7 trillion combinations), a unique index on slug, and a retry when MongoDB returns a duplicate-key error.

How do you count unique visitors without storing IPs?

Salted SHA-256 hash of the IP. It is stable for counting but cannot be reversed to the address without the salt.

Why 302 and not 301?

301 is cached by browsers, so later clicks would not reach the server and analytics would be wrong.

How would you scale it?

Run several API instances behind a load balancer, move the worker to its own process, publish socket events through Redis (Socket.IO Redis adapter), and move analytics to ClickHouse or a Kafka pipeline.

What was the hardest bug?

Write your own here: keep a list while building. Ideas you will probably hit: CORS/socket origin in production, cache not cleared on delete, bots counted as clicks.

9. Build order if you want to learn it by rewriting

Week	Goal
1	Run as is. Read every file. Break things on purpose and fix them. Rewrite models + auth + create-link yourself
2	Rewrite the redirect route with Redis cache. Add the queue and worker
3	Dashboard charts and Socket.IO live feed. Seed data and screenshots
4	k6 load test, find the slow part, fix it, record before/after numbers
5	Deploy, write the README (what broke and what you learned), update the resume

Stretch goals from the guide: Kafka/Redpanda pipeline, ClickHouse analytics, custom domains, link-in-bio page. Only start them after the main version is live.